Predicting Shelter Outcomes

STAT5003 Assignment 2

W01G04

1. Problem Framing & Research Question

Task and motivation

Animal shelters make care and placement decisions with limited space, staff and information. This study addresses a **five-class classification task**: predict Adoption, Transfer, Return, Euthanasia or Other from recorded characteristics of cats and dogs. The unit of analysis is each animal's latest recorded intake in the supplied extract. Predictors are intended to describe information available at admission; their actual recording times are a limitation assessed below.

The motivation is to determine how much useful outcome information these characteristics contain and whether more flexible models improve on simple baselines. Overall accuracy alone is insufficient: a model that mostly recognises common outcomes can overlook animals in rarer outcome groups. We therefore compare macro-F1 and class-specific errors as well as accuracy.

Research question

To what extent can characteristics intended to be available at intake predict the five recorded shelter outcomes for each cat or dog's latest admission, and which fitted model offers the best balance of class-level performance?

Stakeholders and intended use

Intake Triage Officers need evidence to support early assessment and prioritise cases for staff review. Recall for less common outcomes indicates how often relevant cases are detected; precision indicates how often a flag is a false alarm. These predictions could supplement professional judgement, but they do not measure medical deterioration, length of stay or the appropriate treatment for an animal.

Transfer & Foster Coordinators plan rescue-partner and foster placements under capacity constraints. Transfer predictions may help identify cases for placement review; a missed Transfer outcome and a false Transfer prediction have different practical costs. This study evaluates outcome classification, rather than forecasting placement dates or demonstrating reduced shelter stays.

A predicted historical outcome is not a recommendation about what should happen to an animal. The analysis tests associations in historical records, and operational use would first require verified intake-time inputs and an assessment of errors in the intended setting.

2. Dataset & Justification

Source and analytical scope

We analysed the Austin Animal Center Outcomes extract from the City of Austin Open Data Portal (City of Austin, n.d.). The supplied file contains **18,624 records and 16 variables**. Selecting Dog, Puppy, Cat and Kitten yields **17,597 records** for EDA; retaining each animal's latest recorded intake yields **16,831 animals** for modelling. This prevents the same animal appearing in training and assessment data.

Evidence for the dataset criteria

- **Large:** the final sample contains 16,831 animals, exceeding 10,000 even after restricting the scope and removing repeated admissions.
- **Messy:** missing or blank names (38.2%), secondary colours (47.7%) and birth dates (4.7%), inconsistent categorical labels and invalid ages require explicit cleaning. Figure 1 shows the missingness evidence.
- **Multiclass:** five outcomes have substantially different frequencies. Figure 1 shows their distribution and why the evaluation must consider minority classes.

Outcome definition

We consolidate 28 raw outcome labels into five operational groups. Adoption includes adoption variants; Transfer includes transfers out; Return includes reclaimed, redemption, return-to-owner and return-to-habitat records; Euthanasia includes euthanasia variants; Other contains the remaining death and administrative outcomes. Other is consequently a heterogeneous class.

3. EDA & Insights

Data quality and outcome distribution

EDA examines the **17,597 selected records** using their original predictor values. Separate audit and plotting summaries describe missingness, inconsistencies and associations with `Outcome.Group`; selection of the final modelling sample and imputation begin in Data Preprocessing.

The audit finds **676 Animal IDs with repeated records**, but **0 exact duplicate rows**. Repeated Animal IDs may represent separate shelter episodes rather than duplicate records. There are 828 unavailable ages and a further 90 negative ages. These are separate data-quality findings. Negative ages remain in the audit; their conversion to missing values is a later preprocessing decision. No nonblank birth or intake dates failed parsing.

Adoption and Transfer account for 83.4% of selected records. A classifier can therefore achieve substantial accuracy while failing to recognise smaller outcome groups. Missingness is concentrated in secondary colour, name and birth date (Figure 1).

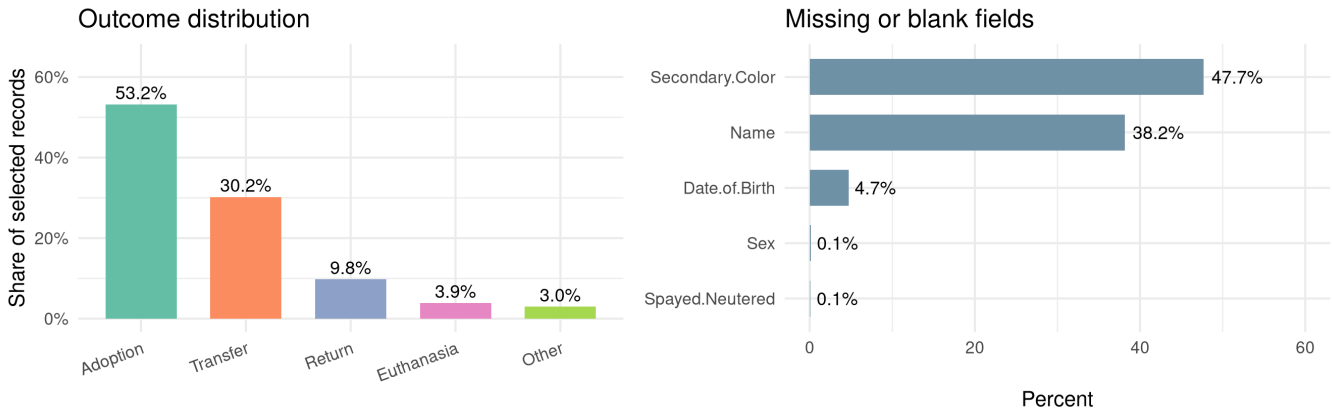


Figure 1. Initial cat-and-dog subset, before selecting one admission per animal.

Type, age and overlapping information

Puppy and Kitten already describe life stage as well as species, so their relationship with observed age indicates overlapping information. The age summaries use only nonnegative observed ages, with missing and negative counts reported separately. Median ages are 1.50 years for Dog and 0.33 for Cat, compared with 0.13 for Puppy and 0.08 for Kitten.

The distribution is severely right-skewed. Animals that were Adopted or Transferred skew noticeably younger, whereas those in the Return or Euthanasia groups skew older. The Return proportions are 16.8% for Dog, 3.9% for Puppy, 6.4% for Cat and 0.6% for Kitten. Alongside the age summaries, these differences motivate representing species and continuous age separately in the modelling dataset.

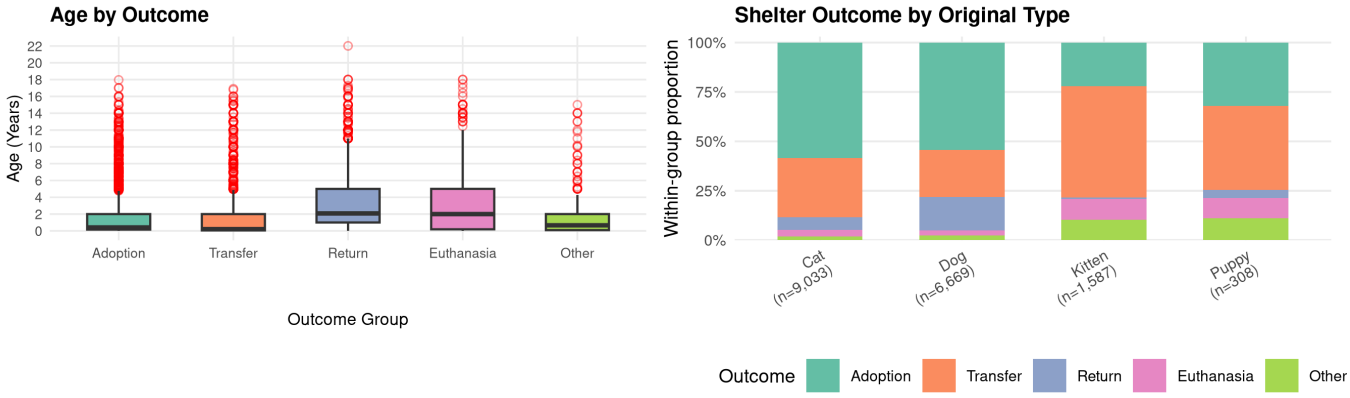


Figure 2. Observed nonnegative age by outcome and outcome proportions within original Type labels.

Breed, colour and coat pattern

There are 198 distinct breed labels recorded; the top eight cover 79.8% of selected records, while 47 labels occur only once. To limit sparse categories, Data Preprocessing retains the eight most common non-Unknown breeds identified in each training partition and groups all remaining or unseen breeds as Other.

The original colour analysis uses a separate plotting copy: primary colours are grouped into base colours, and a non-empty secondary colour or patterned-coat keyword identifies Mixed rather than Pure. White/Cream animals show the highest Return proportion of any colour group, while Blue/Grey animals show a comparatively higher Euthanasia proportion. The additional pattern comparison shows Adoption proportions of 55.3% for Mixed and 45.3% for Pure. These are associations between recorded categories and outcomes.

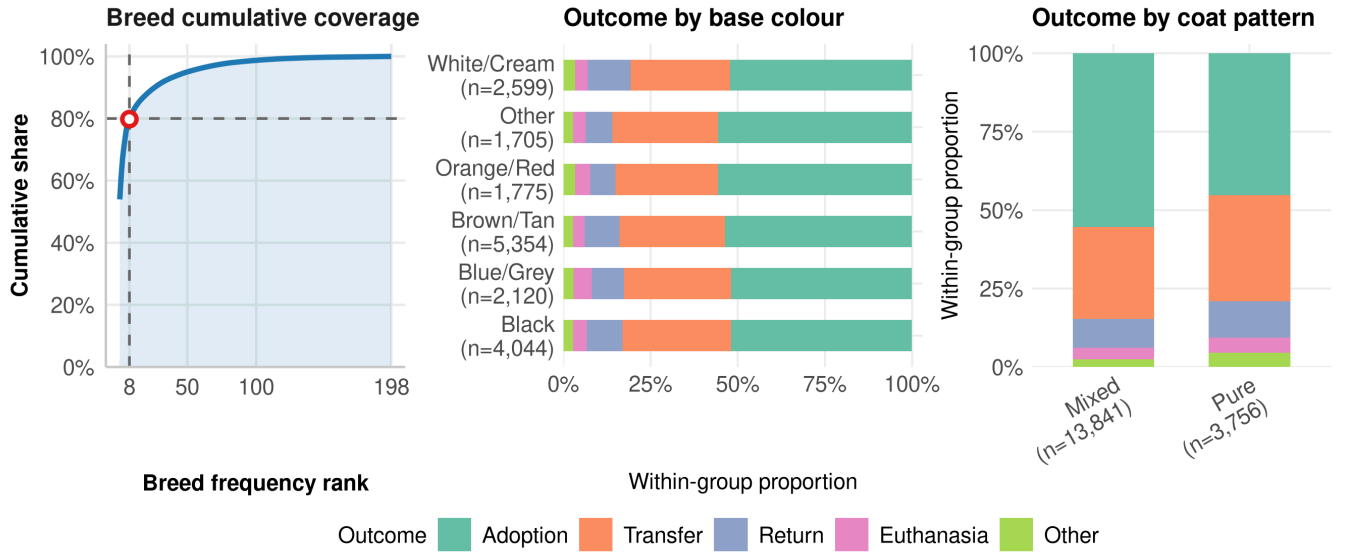


Figure 3. Original breed distribution and colour/pattern outcome proportions; sample sizes are shown with category labels.

Sex, spay/neuter status and name recording

Sex and Spayed/Neutered contain blank values, and Spayed/Neutered additionally has inconsistent capitalisation (Yes and yes both appear and represent the same value). Both issues are corrected in Data Preprocessing; the plots below use the raw values to document what the inconsistency actually looks like before cleaning. Adoption proportions differ even between Yes (79.1%) and yes (98.7%), so these patterns require attention to recording practices.

EDA compares outcome proportions across observed name-recording conditions: non-placeholder text, placeholder text, blank text and NA. This is a missingness audit of Name, not construction of the binary has_Name predictor. Original names are retained; the binary modelling variable is created only in Data Preprocessing. The present subset has no NA names; the observed groups are shown with their sample sizes. Adoption accounts for 70.7% of the Recorded group and 26.5% of the Blank group.

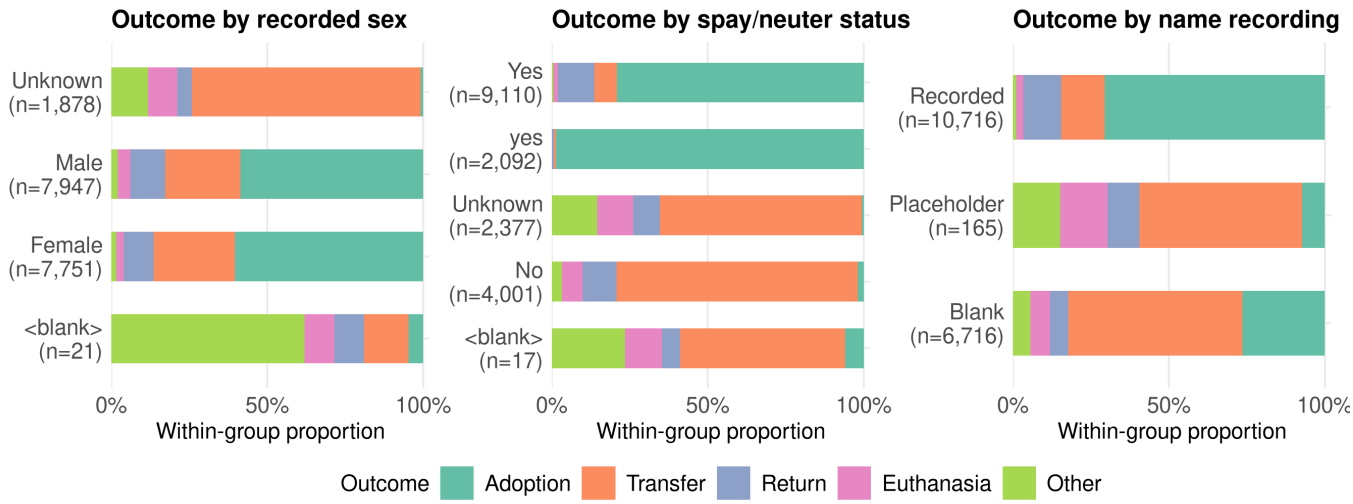


Figure 4. Raw sex/neuter labels and name-recording groups, with group sizes; no imputation or label standardisation in EDA.

Differences between these recording groups may reflect administrative practices or prior ownership information. This comparison cannot establish those explanations or confirm that names were available at intake. Type already contains age-related labels; breed and colour have many categories; date differences include missing and negative values; and Sex, Spayed/Neutered and Name contain recording issues. These findings motivate the explicit preprocessing choices in the next section. The class imbalance supports macro-F1 and per-class recall, while predictive value is assessed using shared cross-validation splits. EDA uses the full selected subset and is not independent performance evidence.

4. Data Preprocessing

We retain the latest intake timestamp for each animal, resolving ties by source row order. This removes 766 additional records without using the outcome to choose an admission. A stratified 80/20 split with seed 5003 gives **13,467 training animals and 3,364 test animals**. Five outcome-stratified folds within training are shared by all models. Animal IDs are used only for grouping and checks.

Predictor	Implemented treatment
Species	Recode Puppy to Dog and Kitten to Cat.
Age at intake	Calendar-day difference between intake and birth, divided by 365; negative values become missing. Impute from the training median within species, with an overall training-median fallback.
Sex; spayed/neutered	Trim and standardise labels; represent missing or unrecognised values as Unknown.
Primary breed	Retain the eight most frequent non-Unknown breeds in the training partition; group the rest and unseen breeds as Other.
Base colour; pattern	Apply fixed text rules to colour fields; pattern indicates a recorded secondary colour or patterned-coat keyword.
Name recorded	Binary indicator; blank, missing and placeholder names are coded absent.

Outcome date/status, euthanasia reason, completed days in shelter, identifiers and administrative timestamps are excluded from predictors. Raw names, dates and text are replaced by their derived features. Imputation and breed grouping are re-estimated within each training fold, then applied to validation rows. Model-specific encoding and age transformations also use the corresponding training rows. Name, neutering status and the colour indicators remain recording-based proxies whose availability at admission requires verification.

5. Model Fitting

Common fitting procedure

Four fitted classifiers use the same eight core predictors and shared five-fold partitions. For each fold, preprocessing is learned from its training rows, the model is fitted to those rows, and predictions are recorded for the validation rows. Combining these predictions gives one out-of-fold prediction per training animal. Hyperparameters and feature changes are evaluated in Section 6; model comparison is reported in Section 7. The folded panels below contain the actual fitting source; each model's Section 6 panel contains its tuning, prediction and result-export code. Tables are calculated from those workflows' saved outputs. XGBoost was rerun for this report; the other three models use existing outputs checked against the available predictions and fold summaries. Rendering the report does not retrain the model grids. The decision-tree model is excluded from this review because its fitted results are not yet available.

Majority class (baseline)

The common reference baseline predicts the most frequent training class for every animal, without using predictors. It is **Adoption** in each training fold. The majority class accounts for 52.40% of the full training partition, while the smallest class, Other, accounts for 2.84%.

On the training labels, this rule achieves accuracy 0.524 and weighted-F1 0.360, but macro-F1 only 0.138 and balanced accuracy 0.200. This illustrates why prevalence-weighted scores can look substantial even when four outcomes are never predicted. The comparison table evaluates this baseline on validation folds, selecting the majority from each fold's training labels. It is a reference rule, separate from the four fitted model families and their own initial configurations.

Multinomial logistic regression

The initial M0 model is fitted with `nnet::multinom`, using treatment-coded categories and training-standardised age. It estimates class probabilities through linear predictors for the class log-odds and assigns the class with the largest probability. This provides a relatively interpretable baseline for assessing whether nonlinear models add predictive value. Regularisation, class weights and nonlinear age features are evaluated in Section 6; the resulting M3 model is fitted with `glmnet`.

Random forest

Random Forest combines many decision trees to capture non-linear relationships and reduce the variance of a single tree. We use the same eight predictors as the other models, keeping categorical variables as factors and age on its original scale.

Baseline model. We first fit a Random Forest with default settings: 500 trees and two randomly selected predictors at each split. No class weights are applied. Performance is assessed using macro-F1, balanced accuracy (mean class recall), and overall accuracy.

The original baseline code below fits the full training partition and predicts the holdout. Section 6 compares the baseline and tuned settings using the shared validation folds; its reported CV scores do not come from these holdout predictions.

Radial-basis support vector machine

We fitted a radial-basis-function support vector machine using `e1071::svm()` with one-versus-one multiclass classification (Chang & Lin, 2011). The eight predictors were `Type`, `Sex`, `Spayed.Neutered`, `primary.breed.group`, `Base_Color`, `has_pattern`, `has_Name`, and `Age_Years_imputed`. Each animal contributed its latest recorded intake only. We used the shared outcome-stratified split and five cross-validation folds (seed 5003) described in Section 4.

Within each fold, the common preprocessing estimated species-specific median ages and the eight most frequent breeds from that fold's training data. The SVM then fitted a full one-hot encoder, removed training-constant columns, and standardised age using the training mean and standard deviation. Binary indicators remained 0/1. Internal SVM scaling was disabled to avoid scaling the same inputs twice. Assessment-only categories were represented by an all-zero block, preserving row alignment. No encoding or scaling parameters were learned from validation data.

The baseline used `C = 1`, `gamma = 1/p`, and no class weights, where `p` is the number of retained encoded predictors.

XGBoost

We use XGBoost as a non-linear model for the classification. It benchmarks the non-linear predictive signal extractable beyond the linear baseline. We use the same predictor set as the multinomial baseline, one-hot encoded into a numeric design matrix for xgboost. Continuous Age_Years_imputed remains unstandardised, as tree-based partition algorithms are invariant to monotonic scaling (Hastie et al., 2009).

XGBoost is fitted with `xgboost::xgb.train`, using the five-class `multi:softprob` objective (Chen & Guestrin, 2016). The predicted class has the largest fitted probability. All configurations use **150 boosting rounds**, seed 5003 and no early stopping. **Configuration A is the XGBoost model baseline:** depth 6, learning rate 0.3 and no class weights; the row/column sampling fractions and minimum child weight retain their defaults of 1.

6. Model Improvement

All improvement choices use the shared training folds and mean CV macro-F1. The test partition does not determine weights, feature transformations or hyperparameters in these procedures. The following comparisons show both the improvement attempted and its measured effect.

Multinomial logistic regression

Starting from the M0 baseline, M1 adds inverse-frequency observation weights, $w_c = n/(5n_c)$, computed from training labels. M2 adds a grouped elastic-net penalty. M3 replaces linear age with a three-degree-of-freedom natural spline of log(1 + age), adds a Dog × standardised log(1 + age) interaction, and includes an age-missing indicator. This allows age relationships to vary across species while controlling unstable coefficients.

For M2 and M3, $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$ is compared along a path of up to 40 regularisation strengths. The path is generated from the full training partition; fold models estimate preprocessing and coefficients separately. After finding the best α, the largest λ within one standard error of its best mean macro-F1 is retained. The selected M3 uses $\alpha = 0.5$, $\lambda \approx 0.026$. Grouped shrinkage can remove encoded columns across classes; it does not necessarily remove a whole original categorical variable.

Table 1. Logistic model improvement, means over the shared five folds.

Specification	CV macro-F1	Balanced accuracy	Accuracy
M0 multinom	0.368	0.386	0.792
M1 multinom + weights	0.428	0.487	0.657
M2 glmnet + weights	0.437	0.496	0.687
M3 glmnet + weights + age features	0.441	0.521	0.624

M3 raises mean macro-F1 from 0.368 to 0.441, while reducing accuracy from 0.792 to 0.624. This reflects greater attention to uncommon outcomes rather than a uniform improvement in prediction.

Random forest

We tune the number of trees (`num.trees` : 100, 300, 500) and the number of predictors considered at each split (`mtry` : 2, 4, 8). All other settings remain at their defaults. Each combination is evaluated using the shared five-fold cross-validation data, and the combination with the highest mean macro-F1 is selected. Test results are not used to choose the parameters.

Table 2. Random forest: baseline and selected configuration on the same five folds.

Configuration	Trees	mtry	CV macro-F1	Balanced accuracy	Accuracy
Unweighted baseline	500	2	0.419	0.417	0.804
Selected by CV	500	4	0.484	0.462	0.800

The selected **500-tree, mtry = 4** model increases mean macro-F1 from 0.419 to 0.484. No class weights are used.

Radial-basis support vector machine

A grid compared `C` in {0.1, 1, 10}, gamma multipliers in {0.25, 1, 4} applied to `1/p`, and unweighted versus inverse-frequency-weighted training: 18 configurations and 90 fold fits. Inverse weights were calculated from the current training partition as `n / (K * n_class)`, so their mean per observation was one. We selected the configuration with the highest mean fold macro-F1, breaking ties by lower fold standard deviation and then lower cost and gamma multiplier.

The selected configuration was **C = 10, gamma = 4/p, weighting = none**. Its mean CV macro-F1 was **0.456**, compared with **0.349** for the baseline, a change of **+0.106**.

Table 3. SVM: baseline, selected model and best weighted alternative on the same five folds.

Configuration	C	Gamma	CV macro-F1	Balanced accuracy	Accuracy
Unweighted baseline	1	1/p	0.349	0.377	0.795
Selected by CV	10	4/p	0.456	0.440	0.800
Best weighted candidate	1	4/p	0.452	0.515	0.668

The best inverse-weighted configuration had mean CV macro-F1 **0.452**, versus **0.456** for the selected model. A small difference in mean macro-F1 does not establish superiority.

XGBoost

For configurations B and C, each training row receives the weight n / (K × n_c) (training rows n, classes K = 5, rows in the row's class n_c), passed to `xgb.DMatrix`. Every class then carries the same total weight, n / K, in the training loss. Weights are computed from the training rows of each fold (and from the full training set for the final model) only.

Four hyperparameters are tuned by grid search: tree depth (`max_depth` : 4, 6), learning rate (`eta` : 0.05, 0.10), the fraction of columns sampled per tree (`colsample_bytree` : 0.70, 0.85) and the minimum child weight (`min_child_weight` : 1, 3). These control model complexity and the degree of regularisation of the boosted trees (Chen & Guestrin, 2016). For the tuned configurations, the row sampling fraction (`subsample`) is fixed at 0.80 and the number of boosting rounds at 150. Each of the 16 configurations is evaluated on the five shared folds in `cv_data`, and the one with the highest mean macro-F1 is selected. To separate the effect of the weights from the effect of tuning, the results below compare three configurations on the same folds: (A) library defaults without weights, (B) library defaults with balanced weights, and (C) the tuned configuration with balanced weights.

Table 4. XGBoost weighting and tuning on identical folds and 150-round budgets.

Configuration	Macro-F1	Fold SD	Balanced accuracy	Accuracy
A. Unweighted model baseline	0.489	0.016	0.468	0.794
B. Baseline + balanced weights	0.476	0.012	0.515	0.693
C. Tuned + balanced weights	0.492	0.013	0.553	0.685

The selected configuration (`max_depth` = 6, `eta` = 0.05, `colsample_bytree` = 0.85, `min_child_weight` = 3) reached a mean macro-F1 of 0.492 (SD 0.013). Adding balanced weights changed mean macro-F1 from 0.489 (A) to 0.476 (B), and mean accuracy from 0.794 to 0.693. Tuning changed mean macro-F1 from 0.476 (B) to 0.492 (C). Because the selected value is the maximum over 16 candidates, it is the selection criterion and not an independent performance estimate; the held-out result in Section 8 is reported separately.

7. Model Evaluation & Selection

Evaluation metrics

For class c, precision = TP/(TP + FP), recall = TP/(TP + FN), and F1 = 2TP/(2TP + FP + FN), using that class's true positives, false positives and false negatives. Precision and F1 are defined as zero when a class is never predicted. **Macro-F1** is the unweighted mean of the five class F1 scores and is the primary selection

measure. **Balanced accuracy** is mean class recall; overall accuracy is secondary. Weighted-F1 weights each class F1 by prevalence and can conceal poor minority-class performance.

Cross-validated comparison

Table 5 reports the mean of the five fold scores and the SD of macro-F1. Figure 5 pools one out-of-fold prediction per training animal for class-specific recall. All models use the same animal assignments, target labels and folds. The folds are also used for tuning, so these are selection results, not unbiased estimates from nested cross-validation. Fold SD describes variation across these folds; it is not a confidence interval.

Table 5. Shared five-fold validation; macro-F1 is the selection criterion.

Model	Macro-F1 (mean ± SD)	Balanced accuracy	Accuracy
XGBoost	0.492 ± 0.013	0.553	0.685
Random forest	0.484 ± 0.016	0.462	0.800
RBF SVM	0.456 ± 0.012	0.440	0.800
Multinomial logistic (M3)	0.441 ± 0.012	0.521	0.624
Majority class (baseline)	0.138 ± 0.000	0.200	0.524

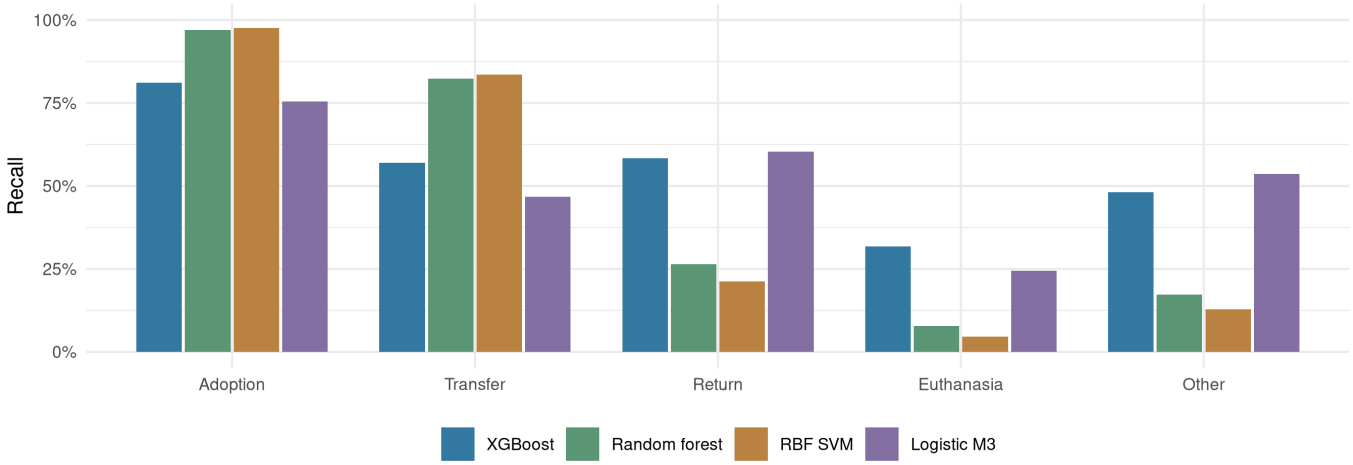


Figure 5. Pooled out-of-fold recall for all four models; each animal is predicted outside its training fold.

XGBoost is selected among the four completed models. Its mean macro-F1 of 0.492 exceeds random forest by 0.008; this small difference is not evidence of a decisive advantage. The practical distinction is its greater recall for Euthanasia and Other. Random forest and SVM achieve about 0.80 accuracy but identify few Euthanasia cases. Logistic M3 has lower macro-F1, although its stronger Return and Other recall makes it informative when missed cases are a priority.

XGBoost's lower accuracy (0.685) is a trade-off: improved attention to rare outcomes comes with more errors among common outcomes. Macro-F1 balances class precision and recall but does not encode a shelter's actual error costs. Staff would need to determine those costs before adopting a model or decision threshold.

8. Final Results & Conclusion

Held-out assessment

After CV selection, the chosen XGBoost configuration is refitted on all 13,467 training animals for the same 150 rounds and evaluated on the 3,364 test animals. Test labels do not determine parameters or the number of rounds in this procedure. Earlier project versions had already reported results on this split; this is a reproduced assessment, not a newly blinded test.

Table 6. Assessment on the same held-out animals.

Model	Macro-F1	Balanced accuracy	Accuracy
Selected XGBoost	0.478	0.534	0.678
Majority class (baseline)	0.138	0.200	0.524

Table 7. Selected XGBoost: per-class held-out performance.

Outcome	Animals	Precision	Recall	F1
Adoption	1764	0.894	0.819	0.855
Transfer	1057	0.890	0.550	0.680
Return	313	0.270	0.559	0.364
Euthanasia	135	0.167	0.289	0.212
Other	95	0.200	0.453	0.277

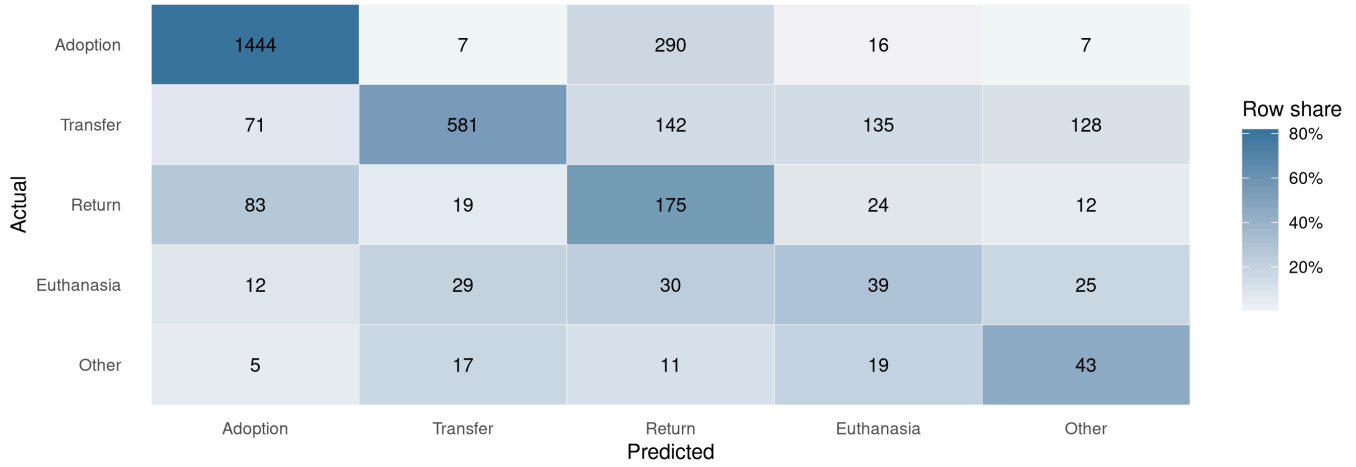


Figure 6. Held-out confusion matrix. Cell labels are animal counts; colour shows the share within each actual class.

Held-out macro-F1 is **0.478**, compared with 0.492 in CV and 0.138 for the majority rule. Euthanasia recall is 28.9% and precision is 16.7%: many actual cases are missed, and many predictions of this outcome are false positives. The model therefore cannot determine an individual animal's care or disposition.

Interpretation and process reflection

Aggregating split gain over one-hot columns identifies **Spayed.Neutered** (37.0% of total gain) and **Age_Years_imputed** (31.7%) as the two largest contributors. Gain describes how the fitted trees split this dataset. Correlated predictors can share importance, and these values neither establish causation nor show that a field was available at intake.

Table 8. Selected XGBoost: share of split gain, aggregated across each predictor's encoded columns.

Predictor	Gain share	Predictor	Gain share
Spayed/neutered	37.0%	Name recorded	5.9%
Age at intake	31.7%	Sex	4.7%
Base colour	7.7%	Species	3.8%
Primary breed	7.0%	Coat pattern	2.1%

The Data Investigation Process connected the audit to implementation: repeated identifiers led to one record per animal, sparse breeds to training-based grouping, and imbalance to macro-F1 and per-class error analysis. Comparing unweighted, weighted and tuned models showed why a plausible adjustment does not necessarily improve the chosen metric. Shared folds allowed a consistent comparison; a stronger evaluation would fix the complete protocol before examining assessment results and reserve a later period for testing.

Answer to the research question and limitations

Recorded animal characteristics provide useful but incomplete information about shelter outcomes. Among the four completed classifiers, tuned weighted XGBoost has the highest mean CV macro-F1. Its held-out macro-F1 is 0.478. Its small advantage over random forest and its lower overall accuracy make the choice dependent on the importance of recognising less common outcomes. The results support further evaluation as an aid to staff review, rather than automated outcome decisions.

The largest practical concern is prediction-time availability. In the logistic sensitivity analysis, excluding spay/neuter status reduces mean CV macro-F1 from 0.441 to 0.369; excluding both that field and name reduces it to 0.362. This dependence does not prove leakage, but it makes verification of intake-time values essential. The same sensitivity checks have not been established for XGBoost.

Other limitations are the heterogeneous Other class, EDA conducted before the split, tuning and ranking on the same folds, and a target population restricted to the latest recorded admission per animal. A random split from one shelter does not establish performance in later periods or other shelters. Future work should verify field timestamps, evaluate a later admission cohort, and assess calibration and stakeholder-defined error costs before operational use.

References

Chang, C.-C., & Lin, C.-J. (2011). LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology, 2*(3), Article 27. https://doi.org/10.1145/1961189.1961199

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). Association for Computing Machinery. https://doi.org/10.1145/2939672.2939785

City of Austin. (n.d.). *Austin Animal Center outcomes* [Data set]. City of Austin Open Data Portal. Retrieved September 20, 2026, from https://data.austintexas.gov/d/gsvs-ypi7

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

AI use statement

OpenAI Codex (GPT-5.6 and GPT-6, OpenAI, https://openai.com/codex/) assisted with R code review, data-processing checks, report organisation, wording and HTML formatting. The report contains AI-assisted text and code; the analysis and submission remain the group’s responsibility.

Reproducibility

Figures are embedded in this HTML and all source panels are collapsed by default. Sections 4–8 show preprocessing, model fitting, tuning, prediction, evaluation and export code alongside the results they support. The accompanying source package retains the complete model files and data. The commands below regenerate each model’s outputs, verify the results and render the report; they are not run during ordinary report rendering. R session information is included for SVM and the XGBoost run; reruns under other package versions can differ numerically.